

# Especificación TLA+ del harness

Máquina de estados de una novela (configuración → planificación → escritura → validación → publicación, con reintentos, crash, reanudación y cambio de un hecho), sus invariantes y propiedades de vivacidad, el resultado de TLC y los contraejemplos que cambiaron el código.

Generado por presentacion/build/build\_annexes.py el 2026-09-25 desde el commit 096b00d. Idioma: español; los ficheros fuente en inglés (código, specs y README técnicos) se reproducen tal cual.

## Resumen de la última ejecución de TLC

Fuente: `formal/tla/tlc-output.txt`

| Dato              | Valor                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| Resultado         | Model checking completed. No error has been found.                                                                       |
| Estados generados | 10.031.846                                                                                                               |
| Estados distintos | 5.492.531                                                                                                                |
| Profundidad       | 127                                                                                                                      |
| Duración          | 08min 17s                                                                                                                |
| TLC               | 2026.09.23.154203                                                                                                        |
| Constantes        | N = 5, SCENES = 3, MAX_SCENE_RETRIES = 2, MAX_CHAPTER_RETRIES = 2, MAX_REPAIR_ROUNDS = 2, MaxCrashes = 1, MaxChanges = 1 |

Los cuatro contraejemplos CE1–CE4 (sección 5) aparecieron en borradores anteriores del modelo y cada uno produjo una regla para el código; la tabla acción → código está en la sección 2.

# Qué se modela, propiedades y correspondencia con el código

Fuente: `formal/tla/README.md`

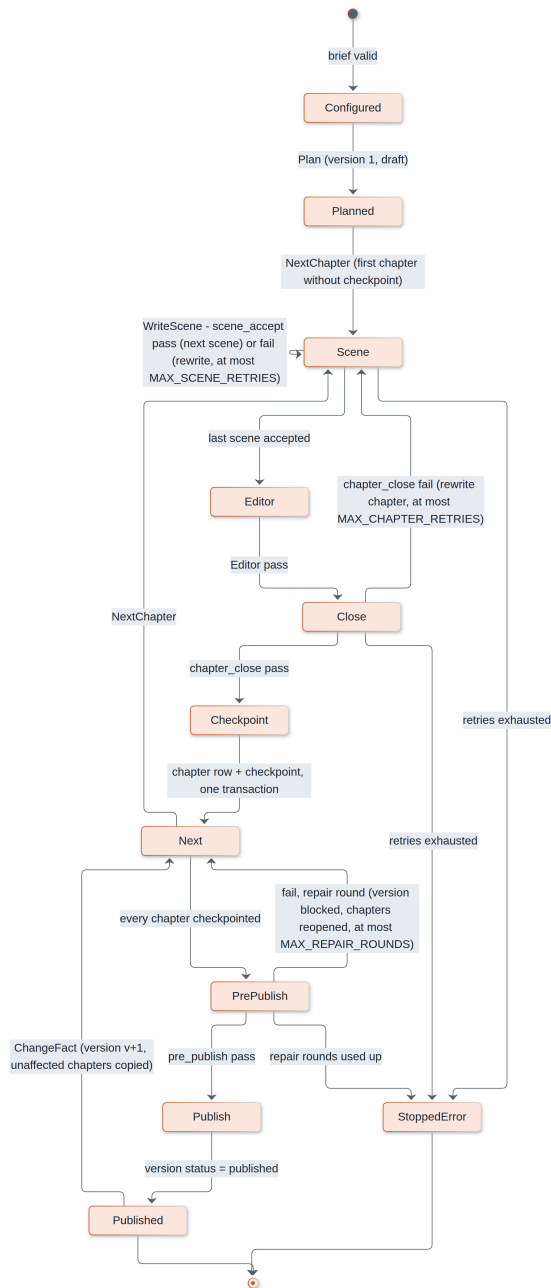
## TLA+ model of the gift-novel harness flow

Spec 013 (block B9 of spec 004, exam items T01–T05). Verification method: model checking, **A** (`docs/verification.md`). TLC runs in development, never per generation.

| File                              | What it is                                                                                        |
|-----------------------------------|---------------------------------------------------------------------------------------------------|
| <code>GiftNovelHarness.tla</code> | The model, plain TLA+                                                                             |
| <code>GiftNovelHarness.cfg</code> | TLC configuration: constants, invariants, properties                                              |
| <code>run-tlc.sh</code>           | Fetches the pinned <code>tla2tools.jar</code> and runs TLC                                        |
| <code>tlc-output.txt</code>       | Output of the last full run                                                                       |
| <code>COUNTEREXAMPLES.md</code>   | Every counterexample TLC found while the model was developed, and the code rule each one produced |

### What is modelled

One novel, from a validated brief to a published version, with crashes and one reader change:



Reading it. Each box is a value of the volatile `pc`. `Crash` is not drawn: it can fire from any state except `Published`, `StoppedError` and `Crashed` itself, wipes the pipeline's memory, and leads to `Crashed`, from which `Resume` reads the database and returns to `Configured` (nothing planned yet), `Published` (latest version already published) or `Next` (a draft or blocked version to finish). `ChangeFact` sends the new version through the same loop: only the chapters it reopened lack a checkpoint, so only they are rewritten.

Durable variables (K1 rows, kept across `Crash`):

| Variable                                                            | K1 row it stands for                                                                                                                                                              |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>vstatus[v]</code>                                             | <code>novel_version.status</code> $\in$ none (no row), draft, blocked, published                                                                                                  |
| <code>changed[v]</code>                                             | <code>novel_version.changed_chapters</code>                                                                                                                                       |
| <code>rows[v][c].origin[v][c]</code> ,<br><code>passed[v][c]</code> | <code>chapter_version(version, chapter)</code> : how many rows, in which version the text was written, whether it passed <code>scene_accept</code> and <code>chapter_close</code> |
| <code>ckpt[v]</code>                                                | <code>checkpoint(version, chapter, status = complete)</code>                                                                                                                      |
| <code>ppOk[v]</code>                                                | the persisted <code>pre_publish_validator_result</code> of the version                                                                                                            |
| <code>chFails[v][c]</code>                                          | <code>chapter_close</code> failures in <code>validator_result</code> since the chapter was opened (CE2)                                                                           |
| <code>repairRounds[v]</code>                                        | repair rounds used by the version, on <code>novel_version</code> (CE4)                                                                                                            |
| <code>factRev</code>                                                | revision of the brief facts ( <code>fact</code> )                                                                                                                                 |

**Volatile variables** (the process's memory, reset by `Crash`): `pc`, `ver`, `cur`, `scene`, `sceneTry`. Scenes are not durable (plan 004, V4): an interrupted chapter restarts from its first scene, so its scene retry counter restarts with it; the budgets that must survive a crash are the chapter and repair counters.

**Ghost variables** (for the properties only): `crashes`, `changes`, `everPublished`, `snap` (the rows of each version as they were when it was published).

**Abstractions.** Validator outcomes are nondeterministic (every validator can pass or fail at every call). The set of chapters a reader change affects, and the set a failed `pre_publish` reopens, are nondeterministic non-empty subsets. Prose, prompts, the judge's scores and the MCP server are not modelled. `ChangeFact` (create version, copy unaffected rows and their checkpoints) is one atomic step: the code must do it in one transaction. Concurrent regenerations are out of scope (spec 004).

Properties

| Name                 | Kind            | Says                                                                                                                                                                                           |
|----------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TypeOK               | invariant       | Every variable stays in its type                                                                                                                                                               |
| NoUnvalidatedPublish | invariant       | A published version has a passed <code>pre_publish</code> result, every chapter checkpointed, and every chapter's stored text passed <code>scene_accept</code> and <code>chapter_close</code>  |
| ResumeNoDupNoLoss    | invariant       | No <code>(version, chapter)</code> is stored twice, and no checkpoint exists without its text                                                                                                  |
| PreviousVersionKept  | invariant       | Every version ever published is still published and its rows are exactly as they were at publication                                                                                           |
| RetriesBounded       | invariant       | Scene retries $\leq$ <code>MAX_SCENE_RETRIES</code> , chapter retries $\leq$ <code>MAX_CHAPTER_RETRIES</code> and repair rounds $\leq$ <code>MAX_REPAIR_ROUNDS</code> , counted across crashes |
| StoreSurvivesCrash   | action property | <code>Crash</code> and <code>Resume</code> change no durable variable, so the set of checkpointed chapters after a resume is exactly the set before the crash (the "no loss" half of resume)   |
| Termination          | liveness        | Every behaviour ends, and stays, in <code>Published</code> or <code>StoppedError</code>                                                                                                        |
| EveryGenerationEnds  | liveness        | Whenever the flow is not terminal (a first generation, a resume, a regeneration), it eventually becomes terminal                                                                               |

Liveness holds under **weak fairness** of every step except `Crash` and `ChangeFact` (`WF_vars(Progress)`), with crashes bounded by `MaxCrashes` and reader changes by `MaxChanges`. Without the bounds, a behaviour that crashes forever would never finish, which is true of the real system too.

How to run

```
formal/tla/run-tlc.sh           # needs Java 11+; downloads tools/tla2tools.jar on first run
```

The jar is the v1.8.0 release asset of `tlaplus/tlaplus`, pinned by SHA-256 in the script, because that tag is a rolling pre-release (the pinned jar reports `TLC2 Version 2026.09.23.154203`, rev 4260e47). `tools/` and `TLC's states/` are git-ignored.

**Configuration** (`GiftNovelHarness.cfg`): `N = 5` chapters, `SCENES = 3` (plan 004, V5), `MAX_SCENE_RETRIES = 2`, `MAX_CHAPTER_RETRIES = 2`, `MAX_REPAIR_ROUNDS = 2` (1 until tuning iteration 1, 2026-09-24; it follows `pipeline.MAX_REPAIR_ROUNDS`), `MaxCrashes = 1`, `MaxChanges = 1`. Nothing had to be reduced.

**Last result** (`tlc-output.txt`, `MAX_REPAIR_ROUNDS = 2`): *Model checking completed. No error has been found.* 10,031,846 states generated, 5,492,531 distinct, search depth 127, 8 min 17 s on 4 workers. With `MAX_REPAIR_ROUNDS = 1` (the configuration before tuning iteration 1) it was 1,247,479 generated, 696,062 distinct, depth 109, about one minute; no new counterexample appeared with the second round.

Mapping: TLA+ action → code

**Confirmed against the code** at 340ede7 (spec 007 pipeline, spec 005 repository). Every function below exists under that name; the pipeline is `backend/app/novel/pipeline.py` (`pipeline.`), the repository `backend/app/bible/repository.py` (`repo.`).

| TLA+ action                     | Code                                                                                                                                                               | State or transition in the code                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Init<br>(pc = "Configured")     | app/interview/service.py ingest_brief                                                                                                                              | a <code>brief</code> row exists and validates ( <code>brief_schema</code> , point hook); no <code>novel_version</code> row                                                                                                                                                                                                                                                    |
| Plan                            | pipeline.plan_novel → persist_plan → record_plan_usage;<br>pipeline._open_version → repo.create_version                                                            | one planner call, <code>check_plan</code> + <code>chronology_problems</code> , at most <code>MAX_REPLANS = 1</code> ; the plan is stored as fact <code>plan.v1</code> ; <code>novel_version(version = 1, status = draft)</code>                                                                                                                                               |
| NextChapter /<br>Resume         | pipeline._chapter_loop via <code>repo.first_incomplete_chapter</code> ; entered from <code>generate</code> ( <code>resume</code> is the same call)                 | the lowest chapter of the version without a <code>complete</code> checkpoint; none left → <code>publish_version</code>                                                                                                                                                                                                                                                        |
| WriteScene                      | pipeline.write_scene → before_scene_accept →<br>run_point(SCENE_ACCEPT)                                                                                            | writer call, then <code>schema_role_output</code> , <code>forbidden_words_scene</code> , <code>no_placeholders</code> ; fail → rewrite with feedback, at most <code>MAX_SCENE_RETRIES = 2</code> (loop counter <b>in memory</b> , see below), then <code>StopRunError</code> → <code>stopped_error</code>                                                                     |
| Editor                          | pipeline.write_chapter → pipeline._edit                                                                                                                            | the editor joins the three scenes (or repairs a reopened chapter, or rewrites a changed one) plus one bounded length adjustment                                                                                                                                                                                                                                               |
| CloseChapter                    | pipeline.close_chapter → before_chapter_close →<br>run_point(CHAPTER_CLOSE)                                                                                        | budget read by <code>_chapter_close_runs</code> from <code>repo.count_chapter_attempts</code> (persisted <code>chapter_close</code> runs since the chapter was last reopened, <b>CE2</b> ); fail → the rejected text goes to <code>repo.record_chapter_attempt</code> and the editor rewrites, at most <code>MAX_CHAPTER_RETRIES = 2</code> , then <code>stopped_error</code> |
| Checkpoint                      | pipeline.checkpoint → repo.save_chapter_and_checkpoint                                                                                                             | <b>one transaction</b> : upsert <code>chapter_version(version, chapter)</code> and <code>checkpoint.status = complete</code> ( <b>CE1</b> ); refused for a published version ( <b>CE3</b> )                                                                                                                                                                                   |
| PrePublish (pass) /<br>Publish  | pipeline.publish_version → before_publish →<br>run_point(PRE_PUBLISH); repo.set_version_status(..., "published")                                                   | <code>schema_brief</code> , <code>brief_coverage</code> , <code>lean_chronology</code> , <code>judge_novel</code> , <code>visual_check</code> ; results persisted; the version becomes <code>published</code> and its rows are never written again                                                                                                                            |
| PrePublish (fail,<br>repair)    | pipeline.publish_version →<br>repo.block_version(..., <code>repair_rounds = repair_rounds + 1</code> ) +<br>repo.set_checkpoint(..., "pending")                    | one transaction: <code>status = blocked</code> , <code>repair_round + 1</code> ( <b>CE4</b> ), the chapters named by <code>chapters_named</code> reopened; <code>_chapter_loop</code> runs again                                                                                                                                                                              |
| PrePublish (fail,<br>exhausted) | pipeline.publish_version → pipeline._stop                                                                                                                          | <code>repair_rounds &gt;= MAX_REPAIR_ROUNDS = 2</code> : the version stays <code>blocked</code> , the novel <code>stopped_error</code> with <code>stop_reason</code> in the version note                                                                                                                                                                                      |
| ChangeFact                      | pipeline.change_fact → repo.update_fact_value →<br>repo.create_version_from(parent, <code>copy_chapters_except = affected</code> )<br>→ <code>_chapter_loop</code> | new version v+1 (draft, parent = latest published) with the unaffected chapters and their checkpoints copied in one transaction; only the affected chapters are rewritten; version v untouched                                                                                                                                                                                |
| Crash                           | process killed (container restart, exception, kill)                                                                                                                | memory lost; SQLite keeps committed transactions                                                                                                                                                                                                                                                                                                                              |
| Resume                          | pipeline.generate / resume → load_plan, _open_version,<br>first_incomplete_chapter                                                                                 | no version → plan; latest published → "already published"; draft or blocked → continue at the first chapter without a complete checkpoint, with chapter and repair budgets read from the database                                                                                                                                                                             |
| Done                            | —                                                                                                                                                                  | terminal: <code>RunResult.status ∈ published, stopped_error</code>                                                                                                                                                                                                                                                                                                            |

**Divergences between the model and the code** (found while confirming the table; recorded in `docs/process/iteraciones.md`):

1. *Scene retry counter in memory.* `write_scene` counts rewrites in a local loop, so a crash restarts the chapter from scene 1 with a fresh scene budget. This is safe and matches the model, where `sceneTry` is volatile: scenes are never checkpointed (plan 004, V4), and the model's `RetriesBounded` across crashes is enforced by the durable chapter-close and repair counters; the extra cost is bounded by the number of crashes.
2. *ChangeFact is not one transaction in the code.* The model takes it as one atomic step (rule 5 below). `change_fact` commits `update_fact_value` (and, for a name, the renames of cast, derived facts and brief) **before** `create_version_from`, which is itself one transaction. A crash between them leaves the fact changed with no version v+1, and `Resume` then finds the latest version published: the change is lost silently. **Fixed in 772f846**: everything from `update_fact_value` to `create_version_from` and the usage rows runs in one `BibleRepository.transaction()` (savepoint-aware).

**Rules the code must follow**, each produced by a counterexample (`COUNTEREXAMPLES.md`):

1. The chapter text and its checkpoint are written in one transaction (**CE1**).
2. The chapter retry count comes from persisted `validator_result` rows, not a loop variable (**CE2**).
3. `chapter_version` is keyed by (`version`, `chapter`) and upserted; a published version's rows are never written (**CE3**).
4. The repair round is stored on `novel_version` and updated with the `blocked` status (**CE4**).
5. Assumed by the model, not found by TLC: `change_fact` creates the new version and its copied rows in one transaction, and publishing is an insert of a new version plus a status change, never an update of an earlier version's rows. Both halves hold in the code (the `change_fact` half since 772f846, divergence 2 above).

# Especificación TLA+

Fuente: [formal/tla/GiftNovelHarness.tla](#)

```
----- MODULE GiftNovelHarness -----
(* ***** *)
(* Gift-novel harness flow (spec 013, block B9 of spec 004). *)
(* *)
(* One novel: Configured (brief valid) -> Planned -> per chapter: scenes *)
(* with scene_accept and bounded rewrite, editor pass, chapter_close with *)
(* bounded chapter rewrite, Checkpoint(c) -> pre_publish with one bounded *)
(* repair round -> Published(v) or StoppedError. A crash may happen at any *)
(* non-terminal step; Resume restarts from the database. A reader change *)
(* on a published version creates version v+1 and regenerates only the *)
(* affected chapters through the same loop. Validator outcomes are *)
(* nondeterministic. *)
(* *)
(* Durable variables stand for K1 rows (they survive Crash); volatile ones *)
(* are the pipeline's memory (reset by Crash); ghost ones exist only for *)
(* the properties. Counterexamples found while developing the model and *)
(* the fixes they caused are in COUNTEREXAMPLES.md (CE1-CE4 below). *)
(* ***** *)
EXTENDS Naturals, FiniteSets

CONSTANTS N, SCENES, MAX_SCENE_RETRIES, MAX_CHAPTER_RETRIES, MAX_REPAIR_ROUNDS,
           MaxCrashes, MaxChanges

Chapters == 1..N
Versions == 1..(MaxChanges + 1)
Terminal == {"Published", "StoppedError"}
Phases == {"Configured", "Planned", "Next", "Scene", "Editor", "Close",
           "Checkpoint", "PrePublish", "Publish", "Published", "StoppedError",
           "Crashed"}
VStatus == {"none", "draft", "blocked", "published"}

Min(S) == CHOOSE x \in S : \A y \in S : x <= y
Max(S) == CHOOSE x \in S : \A y \in S : x >= y

VARIABLES
  \* durable: rows of the authoritative database (K1); survive a crash
  vstatus, \* novel_version.status
  changed, \* novel_version.changed_chapters
  rows, \* number of chapter_version rows stored per (version, chapter)
  origin, \* version in which the stored text was written (0 = none)
  passed, \* the stored text passed scene_accept and chapter_close
  ckpt, \* checkpoint rows: chapters complete per version
  ppOk, \* persisted pre_publish validator_result passed, per version
  factRev, \* revision of the brief facts
  chFails, \* chapter_close failures in validator_result since the chapter
           \* was last opened (CE2: was an in-memory counter)
  repairRounds, \* pre_publish repair rounds used, on novel_version
           \* (CE4: was an in-memory counter)
  \* volatile: memory of the pipeline process; lost on a crash
  pc, ver, cur, scene, sceneTry,
  \* ghost: bookkeeping for the properties only
  crashes, changes, everPublished, snap

durable == <<vstatus, changed, rows, origin, passed, ckpt, ppOk, factRev, chFails,
           repairRounds>>
volatile == <<pc, ver, cur, scene, sceneTry>>
ghost == <<crashes, changes, everPublished, snap>>
vars == <<durable, volatile, ghost>>

Row(v, c) == <<rows[v][c], origin[v][c], passed[v][c]>>

TypeOK ==
  /\ vstatus \in [Versions -> VStatus]
  /\ changed \in [Versions -> SUBSET Chapters]
  /\ rows \in [Versions -> [Chapters -> Nat]]
  /\ origin \in [Versions -> [Chapters -> 0..(MaxChanges + 1)]]
  /\ passed \in [Versions -> [Chapters -> BOOLEAN]]
  /\ ckpt \in [Versions -> SUBSET Chapters]
  /\ ppOk \in [Versions -> BOOLEAN]
  /\ factRev \in 0..MaxChanges
  /\ pc \in Phases
  /\ ver \in 0..(MaxChanges + 1)
  /\ cur \in 0..N
  /\ scene \in 0..SCENES
  /\ sceneTry \in Nat
  /\ crashes \in 0..MaxCrashes /\ changes \in 0..MaxChanges
  /\ everPublished \subseq Versions
  /\ chFails \in [Versions -> [Chapters -> Nat]]
  /\ repairRounds \in [Versions -> Nat]

Init ==
  /\ vstatus = [v \in Versions |-> "none"]
  /\ changed = [v \in Versions |-> {}]
  /\ rows = [v \in Versions |-> [c \in Chapters |-> 0]]
  /\ origin = [v \in Versions |-> [c \in Chapters |-> 0]]
  /\ passed = [v \in Versions |-> [c \in Chapters |-> FALSE]]
```

```

/\ ckpt    = [v \in Versions |-> {}]
/\ ppOk    = [v \in Versions |-> FALSE]
/\ factRev = 0
/\ pc = "Configured" /\ ver = 0 /\ cur = 0 /\ scene = 0
/\ sceneTry = 0
/\ crashes = 0 /\ changes = 0 /\ everPublished = {}
/\ snap = [v \in Versions |-> [c \in Chapters |-> <<0, 0, FALSE>>]]
/\ chFails = [v \in Versions |-> [c \in Chapters |-> 0]]
/\ repairRounds = [v \in Versions |-> 0]

(* Configured (brief valid) -> Planned: version 1 is created as a draft. *)
Plan ==
/\ pc = "Configured"
/\ vstatus' = [vstatus EXCEPT ![1] = "draft"]
/\ changed' = [changed EXCEPT ![1] = Chapters]
/\ ver' = 1 /\ pc' = "Planned"
/\ UNCHANGED <<rows, origin, passed, ckpt, ppOk, factRev, chFails,
    repairRounds, cur, scene, sceneTry, ghost>>

(* Pick the first chapter of the version without a checkpoint. *)
NextChapter ==
/\ pc \in {"Planned", "Next"}
/\ LET todo == Chapters \ ckpt[ver] IN
    IF todo # {}
    THEN /\ cur' = Min(todo) /\ scene' = 1 /\ sceneTry' = 0
        /\ pc' = "Scene"
    ELSE /\ pc' = "PrePublish"
        /\ UNCHANGED <<cur, scene, sceneTry>>
/\ UNCHANGED <<durable, ver, ghost>>

(* Write one scene and run scene_accept (forbidden words, schema). *)
WriteScene ==
/\ pc = "Scene"
/\ /\ /\ scene < SCENES                                     \* pass, more scenes
    /\ scene' = scene + 1 /\ sceneTry' = 0 /\ UNCHANGED pc
  \/ /\ scene = SCENES                                     \* pass, last scene
    /\ pc' = "Editor" /\ UNCHANGED <<scene, sceneTry>>
  \/ /\ sceneTry < MAX_SCENE_RETRIES                       \* fail, rewrite
    /\ sceneTry' = sceneTry + 1 /\ UNCHANGED <<pc, scene>>
  \/ /\ sceneTry = MAX_SCENE_RETRIES                       \* fail, exhausted
    /\ pc' = "StoppedError" /\ UNCHANGED <<scene, sceneTry>>
/\ UNCHANGED <<durable, ver, cur, ghost>>

Editor ==
/\ pc = "Editor" /\ pc' = "Close"
/\ UNCHANGED <<durable, ver, cur, scene, sceneTry, ghost>>

(* chapter_close: length, exact names, judge. *)
CloseChapter ==
/\ pc = "Close"
/\ /\ /\ pc' = "Checkpoint"                                \* pass
    /\ UNCHANGED <<scene, sceneTry, chFails>>
  \/ /\ chFails[ver][cur] < MAX_CHAPTER_RETRIES            \* fail, rewrite chapter
    /\ chFails' = [chFails EXCEPT ![ver][cur] = @ + 1]
    /\ scene' = 1 /\ sceneTry' = 0 /\ pc' = "Scene"
  \/ /\ chFails[ver][cur] = MAX_CHAPTER_RETRIES            \* fail, exhausted
    /\ pc' = "StoppedError"
    /\ UNCHANGED <<scene, sceneTry, chFails>>
/\ UNCHANGED <<vstatus, changed, rows, origin, passed, ckpt, ppOk, factRev,
    repairRounds, ver, cur, ghost>>

(* Checkpoint(c): the chapter_version row and the checkpoint row are *)
(* written in ONE transaction (CE1). The row is keyed by (version, *)
(* chapter) and upserted, never inserted twice, and only a version that is *)
(* not published may be written (CE3). *)
Checkpoint ==
/\ pc = "Checkpoint"
/\ vstatus[ver] # "published"
/\ rows' = [rows EXCEPT ![ver][cur] = 1]
/\ origin' = [origin EXCEPT ![ver][cur] = ver]
/\ passed' = [passed EXCEPT ![ver][cur] = TRUE]
/\ ckpt' = [ckpt EXCEPT ![ver] = @ \cup {cur}]
/\ chFails' = [chFails EXCEPT ![ver][cur] = 0]
/\ pc' = "Next"
/\ UNCHANGED <<vstatus, changed, ppOk, factRev, repairRounds,
    ver, cur, scene, sceneTry, ghost>>

(* pre_publish: brief coverage, Lean, visual check. *)
PrePublish ==
/\ pc = "PrePublish"
/\ /\ /\ ppOk' = [ppOk EXCEPT ![ver] = TRUE]            \* pass
    /\ pc' = "Publish"
    /\ UNCHANGED <<vstatus, ckpt, chFails, repairRounds>>
  \/ /\ repairRounds[ver] < MAX_REPAIR_ROUNDS              \* fail, repair round
    /\ \E R \in (SUBSET Chapters) \ {} :
        /\ ckpt' = [ckpt EXCEPT ![ver] = @ \ R]
        /\ chFails' = [chFails EXCEPT ![ver] =
            [c \in Chapters |-> IF c \in R THEN 0 ELSE @[c]]]
    /\ vstatus' = [vstatus EXCEPT ![ver] = "blocked"]
    /\ repairRounds' = [repairRounds EXCEPT ![ver] = @ + 1]
    /\ pc' = "Next"
    /\ UNCHANGED ppOk
  \/ /\ repairRounds[ver] = MAX_REPAIR_ROUNDS              \* fail, exhausted

```

```

        /\ vstatus' = [vstatus EXCEPT ![ver] = "blocked"]
        /\ pc' = "StoppedError"
        /\ UNCHANGED <<ppOk, ckpt, chFails, repairRounds>>
    /\ UNCHANGED <<changed, rows, origin, passed, factRev,
        ver, cur, scene, sceneTry, ghost>>

(* Published(version). *)
Publish ==
    /\ pc = "Publish"
    /\ vstatus' = [vstatus EXCEPT ![ver] = "published"]
    /\ everPublished' = everPublished \cup {ver}
    /\ snap' = [snap EXCEPT ![ver] = [c \in Chapters |-> Row(ver, c)]]
    /\ pc' = "Published"
    /\ UNCHANGED <<changed, rows, origin, passed, ckpt, ppOk, factRev,
        chFails, repairRounds, ver, cur, scene, sceneTry,
        crashes, changes>>

(* The reader changes a fact of the published version: version v+1 copies *)
(* the unaffected chapters and regenerates the affected ones. *)
ChangeFact ==
    /\ pc = "Published"
    /\ changes < MaxChanges
    /\ LET w == ver + 1 IN
        /\ w \in Versions
        /\ vstatus[w] = "none"
        /\ \E A \in (SUBSET Chapters) \ {{}} :
            /\ vstatus' = [vstatus EXCEPT ![w] = "draft"]
            /\ changed' = [changed EXCEPT ![w] = A]
            /\ rows' = [rows EXCEPT ![w] = [c \in Chapters |->
                IF c \in A THEN 0 ELSE rows[ver][c]]]
            /\ origin' = [origin EXCEPT ![w] = [c \in Chapters |->
                IF c \in A THEN 0 ELSE origin[ver][c]]]
            /\ passed' = [passed EXCEPT ![w] = [c \in Chapters |->
                IF c \in A THEN FALSE ELSE passed[ver][c]]]
            /\ ckpt' = [ckpt EXCEPT ![w] = Chapters \ A]
        /\ ver' = w
    /\ factRev' = factRev + 1
    /\ changes' = changes + 1
    /\ pc' = "Next"
    /\ UNCHANGED <<ppOk, chFails, repairRounds, cur, scene, sceneTry,
        crashes, everPublished, snap>>

(* The process dies: memory is lost, the database is kept. *)
Crash ==
    /\ pc \notin Terminal \cup {"Crashed"}
    /\ crashes < MaxCrashes
    /\ crashes' = crashes + 1
    /\ pc' = "Crashed" /\ ver' = 0 /\ cur' = 0 /\ scene' = 0
    /\ sceneTry' = 0
    /\ UNCHANGED <<mutable, changes, everPublished, snap>>

(* Resume from the database: the latest version, first chapter without a *)
(* complete checkpoint. *)
Resume ==
    /\ pc = "Crashed"
    /\ IF \A v \in Versions : vstatus[v] = "none"
        THEN /\ pc' = "Configured" /\ UNCHANGED ver
        ELSE LET v == Max({x \in Versions : vstatus[x] # "none"}) IN
            /\ ver' = v
            /\ pc' = IF vstatus[v] = "published" THEN "Published" ELSE "Next"
    /\ UNCHANGED <<mutable, cur, scene, sceneTry, ghost>>

Done == pc \in Terminal /\ UNCHANGED vars

Progress == Plan \ / NextChapter \ / WriteScene \ / Editor \ / CloseChapter
    \ / Checkpoint \ / PrePublish \ / Publish \ / Resume

Next == Progress \ / ChangeFact \ / Crash \ / Done

Spec == Init /\ [][Next]_vars /\ WF_vars(Progress)

-----
(* Safety *)

NoUnvalidatedPublish ==
    \A v \in Versions : vstatus[v] = "published" =>
        /\ ppOk[v]
        /\ ckpt[v] = Chapters
        /\ \A c \in Chapters : rows[v][c] >= 1 /\ passed[v][c]

ResumeNoDupNoLoss ==
    \A v \in Versions : \A c \in Chapters :
        /\ rows[v][c] <= 1 \* no chapter stored twice
        /\ c \in ckpt[v] => rows[v][c] = 1 \* no checkpoint without its text

PreviousVersionKept ==
    \A v \in everPublished :
        /\ vstatus[v] = "published"
        /\ \A c \in Chapters : Row(v, c) = snap[v][c]

RetriesBounded ==
    /\ sceneTry <= MAX_SCENE_RETRIES
    /\ \A v \in Versions : repairRounds[v] <= MAX_REPAIR_ROUNDS

```

```
/\ \A v \in Versions : \A c \in Chapters : chFails[v][c] <= MAX_CHAPTER_RETRIES

(* A crash and the resume that follows change nothing in the database.      *)
StoreSurvivesCrash == [][(Crash \ / Resume) => UNCHANGED durable]_vars

(* Liveness *)

Termination == <>[](pc \in Terminal)
EveryGenerationEnds == (pc \notin Terminal) ~> (pc \in Terminal)
=====
```



# Configuración de TLC

Fuente: `formal/tla/GiftNovelHarness.cfg`

```
\* TLC configuration for GiftNovelHarness (spec 013, AC 2 and AC 3).
\* Small model required by T03: 5 chapters x 2 retries. MAX_REPAIR_ROUNDS = 2 since tuning iteration 1.
CONSTANTS
  N = 5
  SCENES = 3
  MAX_SCENE_RETRIES = 2
  MAX_CHAPTER_RETRIES = 2
  MAX_REPAIR_ROUNDS = 2
  MaxCrashes = 1
  MaxChanges = 1

SPECIFICATION Spec

INVARIANTS
  TypeOK
  NoUnvalidatedPublish
  ResumeNoDupNoLoss
  PreviousVersionKept
  RetriesBounded

PROPERTIES
  StoreSurvivesCrash
  Termination
  EveryGenerationEnds
```

## Salida de TLC (sin las líneas de parseo)

```
TLC2 Version 2026.09.23.154203 (rev: 4260e47)
Running breadth-first search Model-Checking with fp 100 and seed 901940712530085344 with 4 workers on 4 cores with 3042MB heap and 64MB
offheap memory [pid: 19391] (Linux 6.18.44-fc-v37 amd64, Ubuntu 21.0.10 64bit, MSBDiskFPSet, DiskStateQueue).
Starting... (2026-09-24 21:19:10)
Implied-temporal checking--satisfiability problem has 2 branches.
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-09-24 21:19:10.
Checking 2 branches of temporal properties for the current state space with 55201 total distinct states at (2026-09-24 21:19:13)
Finished checking temporal properties in 00s at 2026-09-24 21:19:14
Progress(50) at 2026-09-24 21:19:14: 52,535 states generated (52,535 s/min), 29,732 distinct states found (29,732 ds/min), 4,263 states left
on queue.
Checking 2 branches of temporal properties for the current state space with 2083007 total distinct states at (2026-09-24 21:20:14)
Finished checking temporal properties in 20s at 2026-09-24 21:20:34
Progress(79) at 2026-09-24 21:20:34: 1,936,630 states generated (1,884,095 s/min), 1,103,674 distinct states found (1,073,942 ds/min), 124,317
states left on queue.
Progress(86) at 2026-09-24 21:21:34: 4,025,532 states generated (2,088,902 s/min), 2,266,651 distinct states found (1,162,977 ds/min), 184,657
states left on queue.
Checking 2 branches of temporal properties for the current state space with 6611072 total distinct states at (2026-09-24 21:22:34)
Finished checking temporal properties in 01min 04s at 2026-09-24 21:23:38
Progress(93) at 2026-09-24 21:23:38: 6,087,847 states generated (2,062,315 s/min), 3,396,698 distinct states found (1,130,047 ds/min), 182,324
states left on queue.
Progress(100) at 2026-09-24 21:24:38: 8,210,234 states generated (2,122,387 s/min), 4,541,543 distinct states found (1,144,845 ds/min),
137,315 states left on queue.
Progress(127) at 2026-09-24 21:25:38: 10,031,846 states generated, 5,492,531 distinct states found, 0 states left on queue.
Checking 2 branches of temporal properties for the complete state space with 10985062 total distinct states at (2026-09-24 21:25:38)
Finished checking temporal properties in 01min 48s at 2026-09-24 21:27:26
Model checking completed. No error has been found.
  Estimates of the probability that TLC did not check all reachable states
  because two distinct states had the same fingerprint:
  calculated (optimistic): val = 1.4E-6
  based on the actual fingerprints: val = 5.0E-6
10031846 states generated, 5492531 distinct states found, 0 states left on queue.
The depth of the complete state graph search is 127.
The average outdegree of the complete state graph is 1 (minimum is 0, the maximum 31 and the 95th percentile is 2).
Finished in 08min 17s at (2026-09-24 21:27:27)
```

# Contraejemplos encontrados

Fuente: `formal/tla/COUNTEREXAMPLES.md`

## TLC counterexamples found while developing the model

Spec 013, AC 5 (exam item T05). Every entry below is a real violation reported by TLC on an earlier draft of `GiftNovelHarness.tla`, with the configuration in `GiftNovelHarness.cfg` (`N = 5`, `SCENES = 3`, `MAX_SCENE_RETRIES = 2`, `MAX_CHAPTER_RETRIES = 2`, `MAX_REPAIR_ROUNDS = 1`, `MaxCrashes = 1`, `MaxChanges = 1`). TLC searches breadth-first, so each trace is a shortest one. None was planted: each draft modelled the flow as the K1 schema and plan 004 describe it (`chapter_version` and `checkpoint` are two tables; retry counters are the pipeline's loop variables), and TLC showed where that reading breaks.

Four drafts failed; the fifth passes (see `tlc-output.txt`).

| #   | Draft | Invariant violated | Trace length | States explored before the violation | Cause in one line                                               |
|-----|-------|--------------------|--------------|--------------------------------------|-----------------------------------------------------------------|
| CE1 | 1     | ResumeNoDupNoLoss  | 18 states    | 339 distinct                         | Crash between writing the chapter row and the checkpoint row    |
| CE2 | 2     | RetriesBounded     | 21 states    | 549 distinct                         | Chapter retry counter lived in memory; a crash reset it         |
| CE3 | 3     | ResumeNoDupNoLoss  | 46 states    | 3,327 distinct                       | The repair round re-inserted a chapter row that already existed |
| CE4 | 4     | RetriesBounded     | 50 states    | 7,488 distinct                       | Repair-round counter lived in memory; a crash reset it          |

### CE1 — a crash between the chapter row and its checkpoint duplicates the chapter

**Draft 1.** `chapter_close` passing led to two separate actions, `Persist` (insert the `chapter_version` row) and then `Checkpoint` (insert the `checkpoint` row), mirroring the two K1 tables and the two pipeline functions `close_chapter` / `checkpoint`.

Trace (TLC, 18 states).

- Plan — version 1 is a draft.
- NextChapter picks chapter 1; three `WriteScene` steps pass `scene_accept`; `Editor`; `CloseChapter` passes.
- Persist — `rows[1][1] = 1`, but `ckpt[1] = {}`.
- Crash — memory lost, database kept.
- Resume — latest version is 1, a draft; the first chapter without a checkpoint is chapter 1.
- Chapter 1 is written again (3 scenes, editor, close) and `Persist` inserts a second row: `rows[1][1] = 2`.

**Violated.** `ResumeNoDupNoLoss` (`rows[v][c] <= 1`: no chapter stored twice).

**Fix in the model.** `Persist` and `Checkpoint` merged into one action `Checkpoint`: the chapter text and its checkpoint are written atomically.

**Rule for the code.** `checkpoint(version, chapter)` writes the `chapter_version` row and the `checkpoint` row in the same SQLite transaction (`BibleRepository`, one with `conn: block`). A chapter is either fully checkpointed or absent after a crash; `resume` never sees text without a checkpoint.

### CE2 — a crash resets the chapter retry budget

**Draft 2** (CE1 fixed). The chapter-rewrite counter `chTry` was a volatile loop variable of `write_chapter`, reset when the chapter starts. A ghost `chFails[v][c]` counted `chapter_close` failures since the chapter was last opened, independently of memory.

Trace (TLC, 21 states).

- Plan; chapter 1: three scenes, editor, `CloseChapter` **fails** → `chTry = 1`, `chFails[1][1] = 1`, rewrite.
- Crash → `chTry = 0` (memory), `chFails[1][1] = 1` (what really happened).
- Resume; chapter 1 restarts with a fresh budget.
- Two more `CloseChapter` failures → `chTry = 2`, `chFails[1][1] = 3`.

**Violated.** `RetriesBounded` (`chFails[v][c] <= MAX_CHAPTER_RETRIES`): chapter 1 was rewritten three times against a limit of two, and with more crashes the budget is unbounded (each crash buys `MAX_CHAPTER_RETRIES` more model calls).

**Fix in the model.** The volatile `chTry` is removed; the rewrite decision reads the durable `chFails[ver][cur]`, which `Crash` does not touch.

**Rule for the code.** The chapter retry count is derived from persisted `validator_result` rows (point `chapter_close`, this version and chapter, since the chapter was last opened), not from a loop variable. `close_chapter` reads it before deciding rewrite vs. `StoppedError`. The count resets when the chapter is checkpointed and when a repair round reopens it.

### CE3 — the repair round inserts a second row for a chapter

**Draft 3** (CE1, CE2 fixed). A failed `pre_publish` marks the version `blocked`, removes the checkpoints of a non-empty set `R` of chapters and sends them back through the same chapter loop, which ends in the same `Checkpoint` action. `Checkpoint` still *inserted* the row.

Trace (TLC, 46 states).

1. Plan; chapters 1-5 each pass and are checkpointed: `rows[1] = <<1,1,1,1,1>>`, `ckpt[1] = 1..5`.
2. PrePublish **fails**, repair round 1 with `R = {2}`: `vstatus[1] = "blocked"`, `ckpt[1] = {1,3,4,5}`.
3. NextChapter picks chapter 2; it is rewritten and passes; Checkpoint inserts: `rows[1][2] = 2`.

**Violated.** `ResumeNoDupNoLoss` again, this time with no crash at all: the repair path reuses the insert.

**Fix in the model.** Checkpoint sets `rows[ver][cur] = 1` (upsert on the key `(version, chapter)`) and is enabled only when `vstatus[ver] # "published"`.

**Rule for the code.** `chapter_version` has primary key `(version, chapter)`, and checkpoint upserts it (`INSERT ... ON CONFLICT(version, chapter) DO UPDATE`) only while the version is `draft` or `blocked`; the repository refuses any write to a chapter of a `published` version. If spec 005 wants to keep the rejected draft text too (decision D6, "nothing is deleted"), it adds an `attempt` column and a "current" flag; the model's invariant then reads "exactly one current row per `(version, chapter)`". Either way a repair never leaves two current texts for the same chapter.

## CE4 — a crash resets the repair-round budget

**Draft 4** (CE1-CE3 fixed). The repair counter `repairs` was volatile, like `chTry` in CE2. A ghost `repairTotal[v]` counted repair rounds per version.

**Trace (TLC, 50 states).**

1. Chapters 1-5 checkpointed; PrePublish fails; repair round: `repairs = 1`, `repairTotal[1] = 1`, one chapter reopened.
2. The reopened chapter is rewritten and checkpointed.
3. Crash → `repairs = 0`; Resume → version 1 is `blocked`, every chapter checkpointed, so NextChapter goes to PrePublish.
4. PrePublish fails again and, seeing `repairs = 0`, opens a **second** repair round: `repairTotal[1] = 2`.

**Violated.** `RetriesBounded (repairRounds[v] <= MAX_REPAIR_ROUNDS)`: the spec allows one repair round before `StoppedError`.

**Fix in the model.** The volatile counter is removed; the durable `repairRounds[v]` is incremented in the same step that marks the version `blocked` and reopens the chapters.

**Rule for the code.** `novel_version` stores the repair round (`repair_round` column, or the count of `pre_publish` failures in `validator_result` for the version), updated in the same transaction that sets `status = 'blocked'` and resets the reopened chapters' `checkpoint.status`. `publish_version` reads it to choose repair vs. `StoppedError`.

## Draft 5 — no error

With the four fixes, TLC explores the full state space with no invariant or property violated: 1,247,479 states generated, 696,062 distinct, depth 109, about one minute on 4 workers. The run is saved in `tlc-output.txt`.

**Not counterexamples, but checks that the properties are not vacuous** (run on scratch copies, not kept in the repository):

- Reachability: TLC reports states where version 2 is published after a crash and a repair round, where version 1 is published after a crash and a repair, and where a generation stops with an error after its repair round. So the invariants are checked on the paths that matter, not on an empty set.
- Mutation M1 — `ChangeFact` regenerates *in place* (writes version v instead of creating v+1): TLC reports `PreviousVersionKept` violated after 954 distinct states.
- Mutation M2 — NextChapter jumps to `Publish` without `PrePublish`: TLC reports `NoUnvalidatedPublish` violated after 657 distinct states.